

CS62 Final Project

Claremont Colleges Flex Spending Analysis

Jalen DeLoney • Paityn Richardson • Maren Rusk • Nate Wehner

Introduction

At the Claremont Colleges, students receive Flex Dollars each semester which can be spent at campus cafés, restaurants, and convenience stores. The extended hours and quick service of these vendors make them a convenient alternative to using a meal swipe at a dining hall. However, limited dining hall hours and aligned class periods push students to these vendors at similar times, often leading to extended wait times and schedule disruptions.

This project analyzes Fall 2025 Flex Dollar transaction data to model spending activity and offer students a data-driven recommendation of where and when to purchase food with minimal wait times. Users are able to:

- Visualize a vendor's business at different times in the day
- Identify the least busy vendors at any given time of day
- Access a summary of each vendor including peak hours and spending trends

[GitHub Repo](#)

Needfinding

Our initial idea for the project was deliberately open-ended. We acquired the Fall 2025 Flex transaction dataset and were interested in extracting insights that could benefit students at the Claremont Colleges, but we weren't sure how. To explore what data-driven solutions we could offer, we conducted interviews to identify the needs of students as they pertained to Flex, the various vendors, and campus dining as a whole.

We interviewed several Pomona-Pitzer varsity student athletes whose packed schedules and particular dietary needs make them especially sensitive to dining constraints. We asked questions such as:

- How do you typically use your Flex dollars?
- Is the current amount sufficient for the semester?
- When and where do you encounter long lines or congestion?
- How do dining hall hours interact with your practice and class schedules?
- What aspects of the Flex or dining system feel inefficient or unfair?

These interviews revealed several key themes. Many felt that the allocated amounts of Flex were insufficient, especially when making routine purchases like a morning coffee. Some reported long lines and inconvenient rush hours, especially when training schedules forced them to rely on spending Flex rather than getting a meal at a dining hall. In that same vein, dining hall hours were inconvenient and meal periods often overlapped with practices. Students also pointed out

institutional issues like inconsistent meal allocations between the colleges and the “one meal per period” policy.

These insights guided the direction of our project. We decided to focus on predicting vendor busyness and visualizing daily traffic patterns so that students could make informed Flex purchase decisions and avoid congestion. The interviews also motivated the inclusion of vendor overview statistics to reveal insights about spending patterns and highlight patterns that inform discussions regarding adequate Flex allocations.

Numerous concerns were expressed by interviewees that we decided not to attempt to address. For example, students clearly desired institutional policy changes like increased Flex allocations, consistent meal plan policies across the 5Cs, expanding dining-hall hours, increasing staffing, or allowing Flex payments for vending machines, laundry, or printing. These issues fall out of the scope of what could be solved through software. Instead, our project focuses on helping students navigate the existing system more efficiently.

Dataset

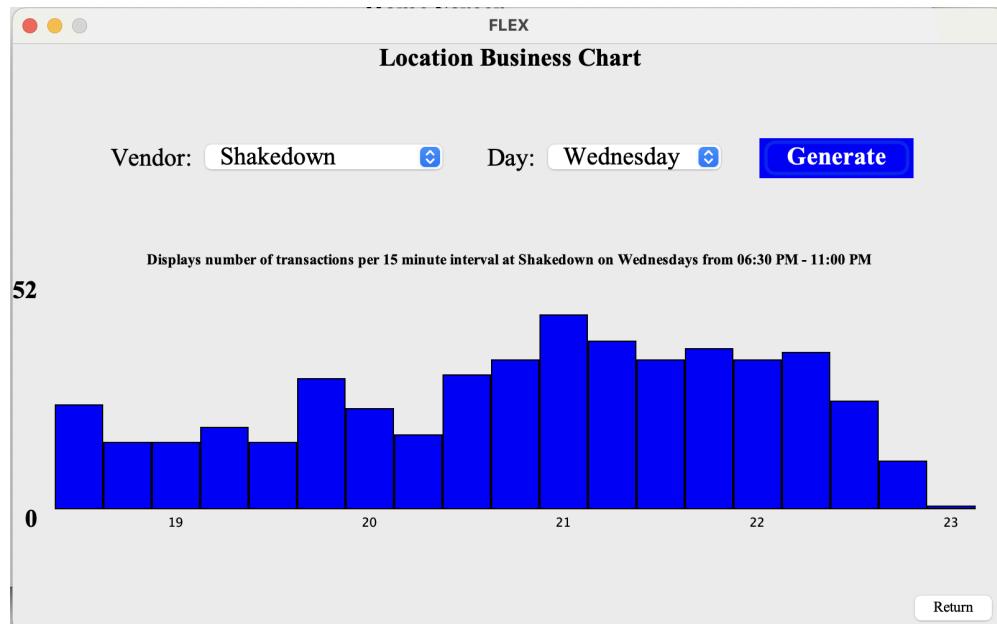
The Fall 2025 Flex transaction dataset was offered by Jeff Myers, a staff member at the Claremont Colleges. It was provided as an Excel spreadsheet detailing over 100,00 Flex transactions from August 5, 2025 up until October 15, 2025. Each row represents a single transaction and includes fields like the date and time, the amount of Flex spent, and the vendor name. We exported the spreadsheet as a csv file to be parsed in Java using `BufferedReader`.

Data Structure

We chose to implement our data structure as nested HashMaps since we wanted to enable fast lookups by numerous different keys like vendor, day of the week, and specific time intervals. Our primary structure maps each vendor to another HashMap keyed by day of the week, which then stores an array representing the 15-minute intervals across a full day. This hierarchy supports constant-time average lookup at each level and makes it efficient to aggregate transaction counts, compute averages, and identify the busiest or least busy intervals.

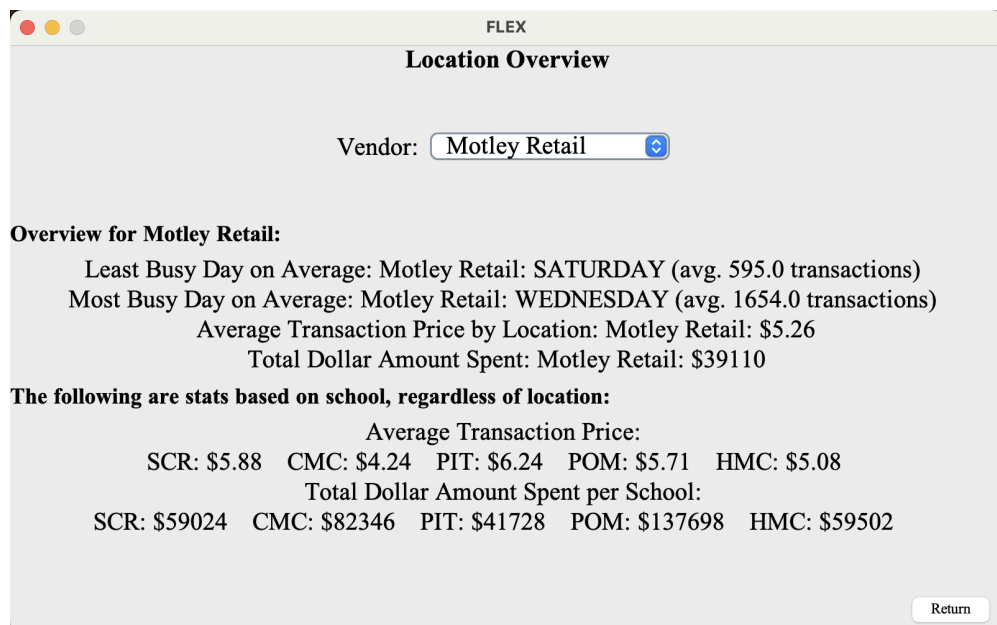
Using HashMap also gave us flexibility in handling vendors with different patterns of activity, allowing us to accommodate different operating hours, for example, and avoided unnecessary memory use for vendors or days with no transactions.

Results



Feature 1: Chart

Displays a histogram with the number of transactions per 15 minute interval at a given vendor on a specified day of the week



Feature 2: Overview

Displays a summary of a specified vendor, including the least busy day on average, most busy day on average, average transaction price, total retail over 6 weeks, as well as the average transaction price and total dollar amount spent per school

FLEX

Least Busy Locations

Day: Time: : PM Number of Locations:

Generate

The following are the least busy open locations for 9:57 PM on Thursday:

1. Coop Fountain: 28
2. Coop Store: 35
3. Shakedown: 36
4. The Hub: 61

[Return](#)

Feature 3: Least Busy

Identifies the specified number of least busy locations for the inputted day of week and time

Analysis

Feature 1: Chart

- Pre-conditions
 - The dataset must be loaded into the TimeData structure (Map<DayOfWeek, Map<String, Map<Integer, Integer>>>).
 - The input vendor must exist in the data for the chosen day.
 - The input day of the week must be valid.
- Time Complexity
 - TimeData has already aggregated transactions into counts per (day, vendor, interval) when the file was read.
 - For the chart, the program:
 - Looks up the day in a HashMap (expected constant time).
 - Looks up the vendor's interval map (expected constant time).
 - Iterates over that vendor's intervals for the day (at most 96 entries, which is a fixed constant).
 - None of this depends on N once the data is loaded so runs in **O(1)** time.
- Space Complexity
 - The feature uses the existing map and array structure for that vendor/day.
 - It only creates constant-size local variables and possibly a fixed-size array for the chart.
 - No additional space grows with N so **O(1)** extra space.

Feature 2: Overview

- Pre-conditions
 - The dataset must be fully loaded into both TimeData and FrequencyData.
 - The input vendor must exist in the dataset.
 - The vendor must have valid entries for at least some days or schools for the summary to be meaningful.
- Edge Cases
 - The vendor has very few transactions, so averages and “busiest/least busy” days may be based on limited data.
- Time Complexity
 - Overview values (least busy day, most busy day, average transaction price, total retail, and per-school statistics) are all computed from the vendor’s existing per-day and per-interval data. The program:
 - Iterates through the fixed 7 days of the week.
 - For each day, iterates through the fixed 96 fifteen-minute intervals.
 - Reads per-school totals from FrequencyData, which stores these values in small maps.
 - All of these loops operate over fixed-size sets so the feature runs in **O(1)** time.
- Space Complexity
 - The feature uses the data already stored for the vendor and only creates small temporary variables while calculating totals and averages.
 - Since no additional structures grow with N, the extra space used is **O(1)**.

Feature 3: Least Busy

- Pre-conditions
 - The dataset must be fully loaded into TimeData.
 - The input day of the week and input time must be valid and convertible to a 15-minute interval.
 - The requested number of locations must be positive.
- Edge Cases
 - No vendors are open at that time and the method returns an empty result with a corresponding message.
 - The user requests more locations than are open and the feature returns all open vendors and notes that fewer were available.
- Time Complexity
 - To find the least busy vendors for a given day and time, the program:
 - Iterates through all vendors in the TimeData map for that day.
 - Performs a constant-time lookup of that vendor’s count for the specified interval.
 - Checks whether the vendor is open at that time.

- Stores eligible vendors in a list, then sorts them by their transaction count.
- Scanning all vendors: $O(N)$.
- Sorting the list of up to N vendors: $O(N \log N)$.
- So the overall runtime is $O(N \log N)$.
- Space Complexity
 - The feature builds a temporary list containing up to N vendors for sorting and ranking. No other large data structures are created.
 - Thus the additional space used is $O(N)$.

Affordance Analysis

This project is intended to benefit all 5C students and affiliates, but it is important to consider results that have potential to disproportionately impact different vendors, students, or workers. For example, the intention of the program to redistribute student transactions to different vendors across the schools could have adverse impacts on the earnings of the vendors, or disruption to the current flow of transaction traffic could end up resulting in even longer waits due to staffing levels and accessibility.

It is particularly necessary to address the power structures that promoting certain on-campus vendors may reinforce. It is well-known that food service workers at the Claremont colleges are underpaid and often overworked, and both student workers and full-time staff have worked tirelessly over the years to organize for improved compensation and workplace protections. If this software had a large impact on spreading peak transaction times throughout the day, there is a chance that already highly pressured workers would lose essential breaks in their workdays that come from natural ebbs and flows in transaction traffic. This brings up the important consideration that structures are likely built around daily transaction trends, with less staffing during hours that are not typically as busy. Thus redistributing these purchasing trends may put more stress on workers who work at less typical hours and may not be considered by the institution.

Finally, there might be reasons we have not considered that on-campus vendors wouldn't want their average transaction price or their total purchase amount disclosed to the public. Although this transaction data was publicly available, sometimes compiling information in such a way that reveals less easily discoverable information could create security risks for the business.

Reflection

Overall, this project took our group about 15 hours of work per person. It aligned mostly with expectations time-wise, but there were a few unexpected twists we came across that ended up taking a bit of time to rework. In particular, we discovered along the way that in order for our output to make sense, we needed to take into account the open and closed hours of each vendor, which varied for every day of the week. This added a layer of complexity because it required us to

incorporate new data structures and manually input lots of data, but was important to keeping our program user-friendly. We ended up using more data structures than we had initially planned on because of pivots like this and created quite a few more classes than expected at the start. However, for the most part we stuck with our original idea and were successfully able to implement each of the three features we proposed, as well as an interactive Graphical User Interface.